AI Assisted Coding
Assignment-14.2

Name: K.Laxman

Ht.No: 2303A51428

Batch: 21

Task Description 1 – (Landing Page Design)

Prompt:

Create a simple modern landing page for a product called "SmartNote" using HTML and CSS.

The page should have:
• A sticky header with the logo "SmartNote" and navigation links (Home, Features, Contact).
• A hero section with a large title, a short description of the product, a "Get Started" button, and a placeholder image beside the text.
• A footer with copyright text and social media links.

Use clean, proper spacing, and a modern color style.

Code:

Run Terminal Help AI assisted Coding

test-1.py | AIAC-Assignment-13.5.py | Release Notes: 1.111.0 | fronted.html | styles.css | Assignment-3.py | student.

fronted.html > html > body > header.sticky-header

```html
<!-- '''Create a simple modern landing page for a product called "SmartNote" using HTML and CSS.

The page should have:
•   A sticky header with the logo "SmartNote" and navigation links (Home, Features, Contact).
•   A hero section with a large title, a short description of the product, a "Get Started" button, and a
placeholder image beside the text.
•   A footer with copyright text and social media links.

Use clean, proper spacing, and a modern color style
to make the landing page visually appealing.''' -->
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>SmartNote</title>
    <link rel="stylesheet" href="styles.css">
</head>
<body>
    <header class="sticky-header">
        <div class="logo">SmartNote</div>
        <nav>
            <a href="#home">Home</a>
            <a href="#features">Features</a>
            <a href="#contact">Contact</a>
        </nav>
    </header>

    <section class="hero">
        <div class="hero-content">
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

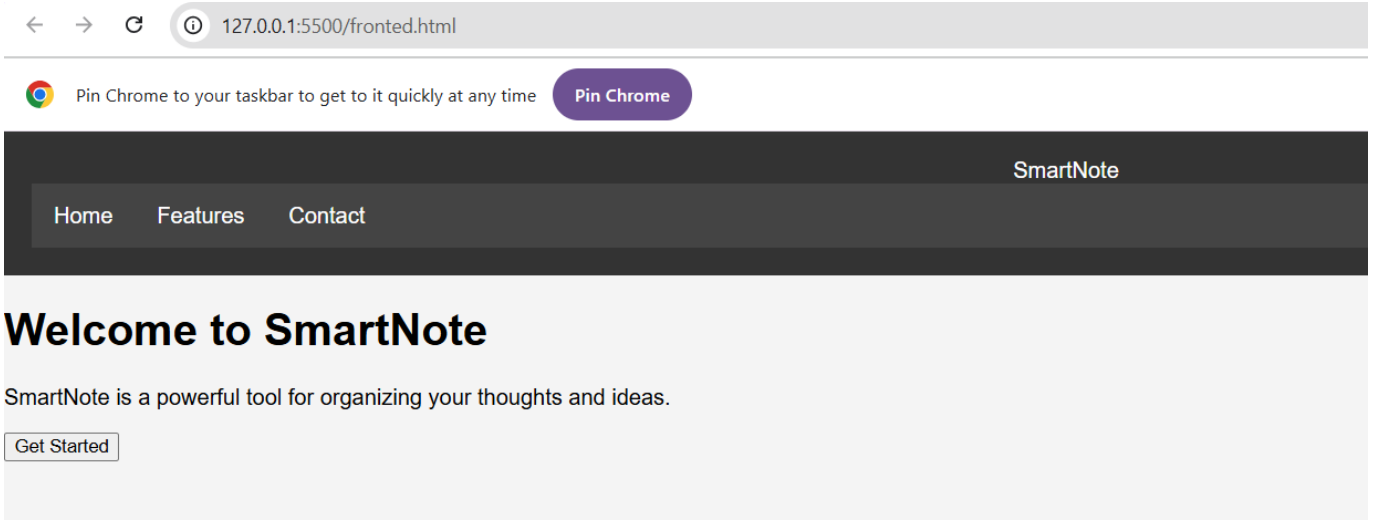
Filter (e.g. text, !exclude, \esca...

test-1.py | AIAC-Assignment-13.5.py | Release Notes: 1.111.0 | fronted.html | styles.css | Assignm

styles.css > header

```css

body {
    font-family: Arial, sans-serif;
    margin: 0;
    padding: 0;
    background-color: #f4f4f4;
}
header {
    background-color: #333;
    color: #fff;
    padding: 20px;
    text-align: center;
}
nav {
    background-color: #444;
    overflow: hidden;
}
nav a {
    float: left;
    display: block;
    color: #fff;
    text-align: center;
    padding: 14px 16px;
    text-decoration: none;
}
nav a:hover {
    background-color: #ddd;
    color: black;
}
main {
    padding: 20px;
}
footer {
    background-color: #333;
    color: #fff;
    text-align: center;
    padding: 10px;
    position: fixed;
```

Output:



Observation:

While creating the SmartNote landing page, I observed how HTML structures the webpage and CSS improves the visual design. The header, hero section, and footer helped organize the content clearly. Using Flexbox made alignment and spacing easier. The sticky header improved navigation while scrolling. The page was made responsive so it works properly on both mobile and desktop screens.

Task Description #2 – Theme Toggle Feature

Prompt:

Add a light and dark theme toggle button to the webpage.

When the button is clicked, switch between light and dark mode.

Use JavaScript to toggle a class on the body element.

Define the colors in CSS and add a smooth transition.

define the html, css, and js code to implement this functionality.

Make sure the theme changes instantly when the button is clicked.

Code:

fronted.html > html > head > style > body

```html
<!-- '''Add a light and dark theme toggle button to the webpage.
When the button is clicked, switch between light and dark mode. Use JavaScript to toggle a class on the body element
Define the colors in CSS and add a smooth transition.
define the html, css, and js code to implement this functionality. Make sure the theme changes instantly when the bu
for the smartNote application with proper theming and website buttons and the style and js in same file'' -->
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>SmartNote</title>
    <style>
        /* Define light and dark theme colors */
        body {
            transition: background-color 0.3s, color 0.3s;
        }
        .light-theme {
            background-color: #ffffff;
            color: #000000;
        }
        .dark-theme {
            background-color: #121212;
            color: #ffffff;
        }
        /* Style for the toggle button */
        .theme-toggle-button {
            position: fixed;
            top: 10px;
            right: 10px;
            padding: 10px 20px;
```

Output:

Welcome to SmartNote

Your smart note-taking application.

Welcome to SmartNote

Your smart note-taking application.

Observation:

In this task, I observed how JavaScript can change the appearance of a webpage. The theme toggle button switches between light and dark mode by changing CSS classes. This makes the interface more user friendly. CSS controls the colors while JavaScript handles the interaction.

Task Description #3 – User

Feedback Form

Prompt:

Create a feedback form with fields: Username, Rating (1–5), and Comments.

Add a submit button and use JavaScript to check if required fields are filled.
If fields are empty, show an error message.
If the form is correct, display a success message.
provide the complete HTML, CSS, and JavaScript code for this form.
Style the form with CSS and keep the layout simple and responsive.

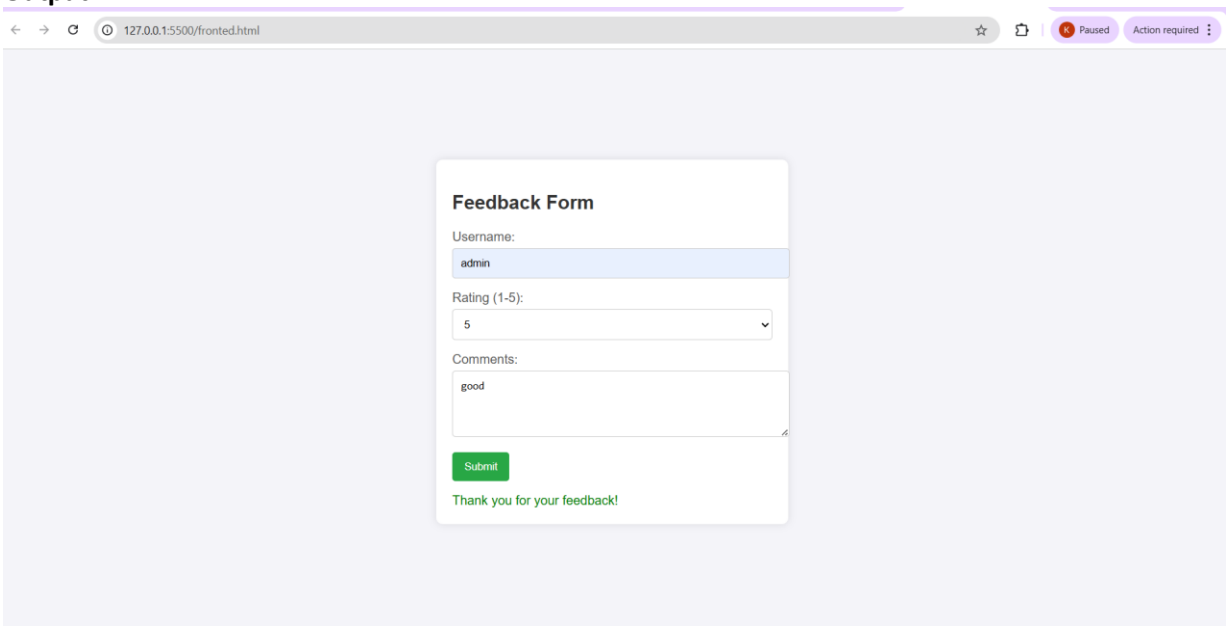
Code:

fronted.html > html > head

```html
<!-- Create a feedback form with fields: Username, Rating (1-5), and Comments.

Add a submit button and use JavaScript to check if required fields are filled. If fields are empty, show an error me
If the form is correct, display a success message.
provide the complete HTML, CSS, and JavaScript code for this form. Style the form with CSS and keep the layout simpl
all in one file.
 -->
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Feedback Form</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            background-color: #f4f4f4;
            display: flex;
            justify-content: center;
            align-items: center;
            height: 100vh;
            margin: 0;
        }
        .feedback-form {
            background-color: #fff;
            padding: 20px;
            border-radius: 5px;
            box-shadow: 0 0 10px rgba(0, 0, 0, 0.1);
            width: 100%;
            max-width: 400px;
        }
        .feedback-form h2 {
            margin-top: 0;
        }
        .feedback-form label {
            display: block;
```

```html
fronted.html > html > head
<html lang="en">
<body>
    <div class="feedback-form">

        <label for="comments">Comments:</label>
        <textarea id="comments" name="comments" rows="4"></textarea>

        <button type="submit" onclick="submitFeedback()">Submit</button>
        <div class="message" id="message"></div>
    </div>

    <script>
        function submitFeedback() {
            const username = document.getElementById('username').value.trim();
            const rating = document.getElementById('rating').value;
            const messageDiv = document.getElementById('message');

            if (!username || !rating) {
                messageDiv.textContent = 'Please fill in all required fields.';
                messageDiv.style.color = 'red';
                return;
            }

            messageDiv.textContent = 'Thank you for your feedback!';
            messageDiv.style.color = 'green';
        }
    </script>
</body>
</html>
```

Output:



Observation:

While creating the feedback form, I observed how forms collect user input on a webpage. JavaScript was used to check whether required fields are filled. It displays error or success messages without reloading the page. CSS helped make the form clean and easy to use.

Task Description #4 – Image

Gallery With JS

Prompt:

Create a small image gallery using HTML, CSS, and JavaScript.

Display one image at a time with Next and Previous buttons.
Store image links in a JavaScript array and change the image when buttons are clicked.
provide html, css, and js code for the gallery.
Style the gallery so it is centered and responsive.

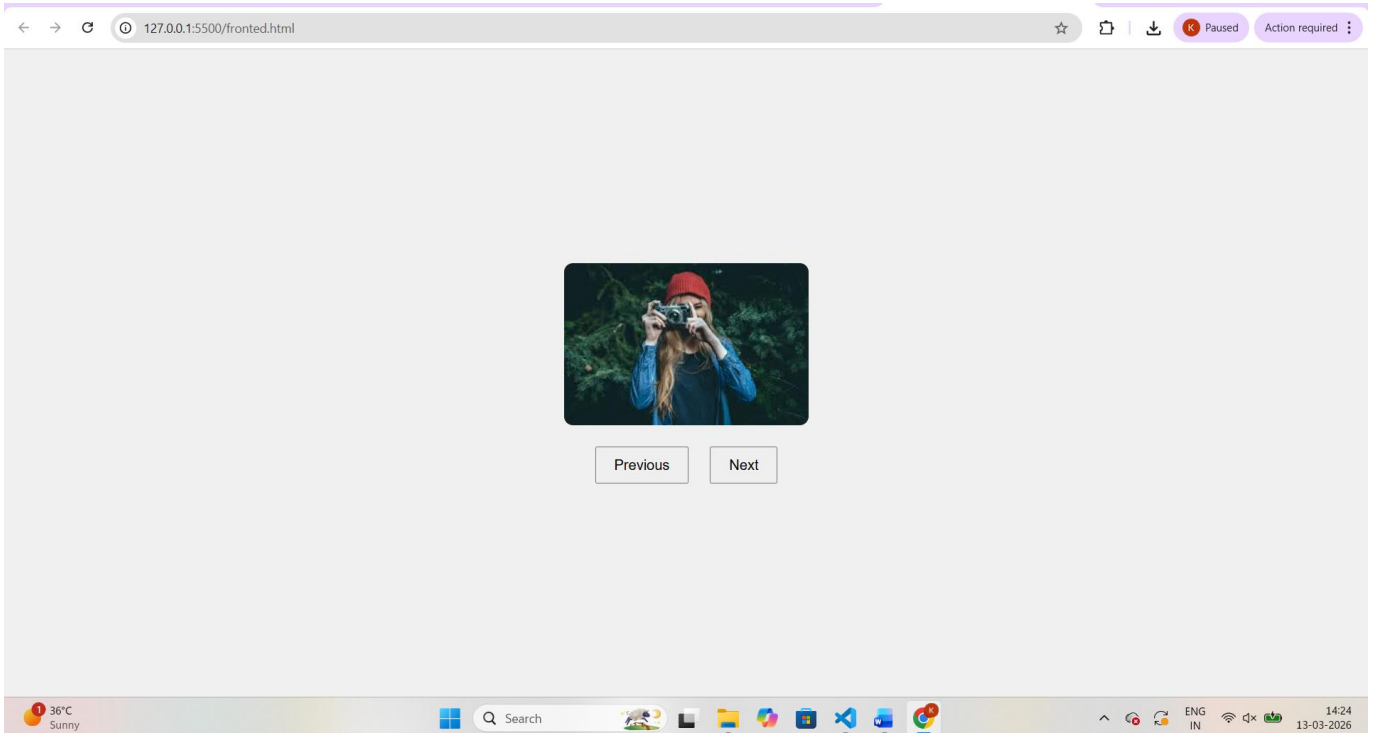
Code:

fronted.html > html > body > div.gallery > img#gallery-image

```html
<!-- Create a small image gallery using HTML, CSS, and JavaScript.

Display one image at a time with Next and Previous buttons.
Store image links in a JavaScript array and change the image when buttons are clicked. provide html, css, and js coc
Style the gallery so it is centered and responsive
 -->
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Image Gallery</title>
    <style>
        /* CSS for the image gallery */
        body {
            display: flex;
            justify-content: center;
            align-items: center;
            height: 100vh;
            margin: 0;
            background-color: #f0f0f0;
        }
        .gallery {
            text-align: center;
        }
        .gallery img {
            max-width: 100%;
            height: auto;
            border-radius: 10px;
        }
        .buttons {
            margin-top: 20px;
        }
        .buttons button {
            padding: 10px 20px;
            font-size: 16px;
            margin: 0 10px;
```

```html
<html lang="en">
<body>

    <script>
        // JavaScript for the image gallery
        const images = [
            'https://encrypted-tbn3.gstatic.com/images?q=tbn:ANd9GcQ_Zuz3haRHrSz0f3bnMlUTGa14Qc7Z5LLQ3-l04P98hv9CMX(
            'https://via.placeholder.com/600x400?text=Image+2',
            'https://via.placeholder.com/600x400?text=Image+3',
            'https://via.placeholder.com/600x400?text=Image+4',
            'https://via.placeholder.com/600x400?text=Image+5'
        ];

        let currentIndex = 0;
        const galleryImage = document.getElementById('gallery-image');
        const prevBtn = document.getElementById('prev-btn');
        const nextBtn = document.getElementById('next-btn');

        function updateImage() {
            galleryImage.src = images[currentIndex];
        }

        prevBtn.addEventListener('click', () => {
            currentIndex = (currentIndex - 1 + images.length) % images.length;
            updateImage();
        });

        nextBtn.addEventListener('click', () => {
            currentIndex = (currentIndex + 1) % images.length;
            updateImage();
        });

        // Initialize the gallery with the first image
        updateImage();
    </script>
</body>
```

Output:



Observation:

In this task, I observed how JavaScript can control images on a webpage. The images are stored in an array and change when the Next or Previous button is clicked. This makes the gallery interactive. CSS helps align the images and keeps the layout responsive.

Task Description #5 – Notification Banner

Prompt:

Create a notification banner at the top of the webpage.

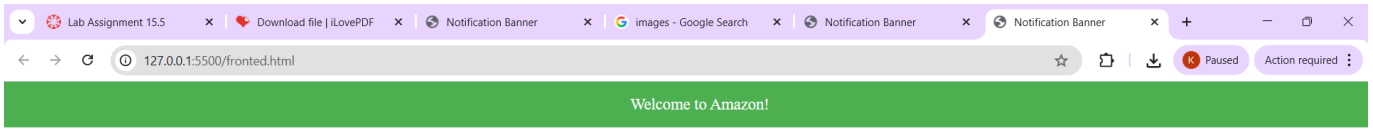
Show a message like "Welcome to SmartNote".
Add a close (X) button to hide the banner.
Use JavaScript to show and hide the banner.
provide html, css, and js code for the banner.
Style the banner using CSS and add a smooth transition.

Code:

```
fronted.html > html > head > style > #close-btn
<!-- Create a notification banner at the top of the webpage.

Show a message like "Welcome to amazon". Add a close (X) button to hide the banner.
Use JavaScript to show and hide the banner. provide html, css, and js code for the banner.
Style the banner using CSS and add a smooth transition.
 -->
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Notification Banner</title>
    <style>
        /* CSS for the notification banner */
        #notification-banner {
            position: fixed;
            top: 0;
            left: 0;
            width: 100%;
            background-color: #4CAF50; /* Green background */
            color: white; /* White text */
            text-align: center; /* Centered text */
            padding: 15px; /* Some padding */
            font-size: 18px; /* Larger font size */
            z-index: 1000; /* Ensure it stays on top */
            transition: top 0.5s ease; /* Smooth transition for hiding/showing */
        }
        #close-btn {
            position: absolute;
            right: 20px;
            top: 15px;
            cursor: pointer;
            font-weight: bold;
        }
    </style>
</head>
```

```html
<head>
    <style>
        #close-btn {

        }
    </style>
</head>
<body>

    <!-- Notification Banner -->
    <div id="notification-banner">
        Welcome to Amazon!
        <span id="close-btn">&times;</span>
    </div>

    <script>
        // JavaScript to handle the close button functionality
        document.getElementById('close-btn').addEventListener('click', function() {
            var banner = document.getElementById('notification-banner');
            banner.style.top = '-100px'; // Move the banner up to hide it
        });
    </script>
</body>
</html>
```

Output:



Observation:

While implementing the notification banner, I observed how JavaScript can control the visibility of elements on a webpage. The banner appears at the top when the page loads. When the close button is clicked, JavaScript adds a CSS class that hides the banner with a smooth transition. CSS helps style

the banner and make it visually clear for users.